Spin simulation notebook — user manual

This folder holds everything you need to simulate a magnetic material with the notebook `spin_simulation.ipynb`. You do not need to know how to program, and you do not need to install the repository: the notebook carries its own code.

In one sentence: you open the notebook in Kaggle or Google Colab, change the numbers in **one cell**, press **Run all**, and download the results.

1. What the notebook does

It simulates spins (tiny magnets) sitting on the sites of a crystal. For every temperature, from hot to cold, it lets the spins reach equilibrium and then measures:

- the **energy** E,
- the **magnetization** M (how aligned the spins are),
- the **specific heat** Cv and the **susceptibility** χ (their peaks mark the phase transition),
- the full **configuration**: the direction of every spin, in every copy of the system.

You can simulate several **states** at once. A state is one set of parameters of the Hamiltonian, for example ”no field” and ”with field”. Each state runs many independent copies of the system, called **replicas**, and the results are averaged over them.

2. Where to run it

Where	What you need	Speed
Kaggle (recommended)	A free account, verified by phone (needed for GPUs)	Fast: two T4 GPUs
Google Colab	A Google account	Fast: one T4 GPU
Your own computer	Linux (or Windows with WSL) and an NVIDIA GPU	Depends on your GPU
Any of them without a GPU	Nothing more	**Very slow**: only for tiny tests

The notebook finds the hardware by itself: it uses every NVIDIA GPU it sees, and the CPU when there is none. The GPU must have compute capability 7.0 or more (T4, V100, A100, H100, RTX 20xx and later). Older GPUs such as the P100 are not supported; the notebook then falls back to the CPU.

2.1 Kaggle, step by step

1. Go to `https://www.kaggle.com`, sign in, and click **Create → New Notebook**.
2. In the menu choose **File → Import Notebook** and upload `spin_simulation.ipynb`.
3. In the right panel, open **Session options** and set **Accelerator** to **GPU T4 x2**.
4. Edit cell 1 (see section 3).

5. Click **Run all**.
6. When it finishes, the results are in the **Output** section of the right panel, in the folder `simulation_output`. Download them from there.

A Kaggle session lasts at most 12 hours, and the free GPU quota is about 30 hours per week.

2.2 Google Colab, step by step

1. Go to `https://colab.research.google.com` and choose **Upload**, then select `spin_simulation.ipy`
2. Menu **Runtime → Change runtime type → T4 GPU → Save**.
3. Edit cell 1.
4. Menu **Runtime → Run all**.
5. To download the results as one file, add a new cell at the end with the line below, run it, and download `simulation_output.zip` from the folder icon on the left:

```
!zip -r simulation_output.zip simulation_output
```

2.3 Your own computer

```
pip install torch triton numpy matplotlib jupyter
jupyter notebook spin_simulation.ipynb
```

Then edit cell 1 and choose **Run → Run All Cells**. The results are written next to the notebook.

3. The only cell you edit: cell 1

Cell 1 is divided into eight blocks. Each line has a short comment next to it. Change only the values after the = sign, and keep the quotation marks, brackets and commas.

3.1 Type of Hamiltonian — `MODEL`

Value	Spins	Example
`"ising"`	up or down (±1)	`MODEL = "ising"`
`"xy"`	arrows that turn in a plane	`MODEL = "xy"`
`"heisenberg"`	arrows that turn in 3D	`MODEL = "heisenberg"`

3.2 The Hamiltonian

The energy the notebook uses is the sum of four terms:

- **Exchange** `J`: neighbouring spins want to be parallel (J > 0, ferromagnet) or opposite (J < 0, antiferromagnet). You give one value per **shell** of neighbours: shell 1 = nearest neighbours, shell 2 = next nearest, and so on.
- **DMI** `D`: interfacial Dzyaloshinskii–Moriya interaction. It twists the spins and creates skyrmions. It acts on the nearest-neighbour bonds lying in the x–y plane.
- **Cubic anisotropy** `K1`, `Ks`, `K2`: favours some directions of the crystal. `Ks` is the value on the **surface** (sites with fewer neighbours than in the bulk); if you leave it out, it equals `K1`.
- **Field** `H`: an external magnetic field. Its direction is given by two angles in degrees: `H_theta` (0 = along +z, the normal of a film or disc; 90 = in the plane) and `H_phi` (the angle in the x–y plane, 0 = along +x).

`D`, `K1`, `Ks` and `K2` only exist for `"heisenberg"`. Ising and XY use `J` and `H` only.

3.3 The states — `STATES`

Each state is one line `dict(...)` inside the brackets, ending with a comma. Write only the parameters you need: **anything you leave out is zero**.

```
STATES = [
    dict(name="no field",   J={1: 1.0}),
    dict(name="with field", J={1: 1.0}, H=0.2, H_theta=0.0),
    dict(name="frustrated", J={1: 1.0, 2: -0.3}),
]
```

- `name` is the label used in the plots; every state needs a different name.
- `J={1: 1.0, 2: -0.3}` means J = 1 for nearest neighbours and J = −0.3 for the second shell.
- To switch a state off without deleting it, put `#` at the start of its line.

3.4 Units — `UNITS`

Value	Energies	Temperature
`"reduced"`	in units of your J (J = 1)	same units, k_B = 1
`"meV"`	in meV	in kelvin

3.5 Crystal and box — `CRYSTAL`, `BOX`, `PERIODIC`

Dimension	Crystals	Box
2D	`"square"`, `"triangular"`, `"honeycomb"`, `"kagome"`	`BOX = [Lx, Ly]`
3D	`"sc"`, `"bcc"`, `"fcc"`	`BOX = [Lx, Ly, Lz]`

- The box is counted in **unit cells**. Equal sizes give a square (or cubic) box; different sizes give a rectangular one. bcc has 2 sites per cell and fcc has 4.

- `PERIODIC` has one `True` or `False` per direction. `True` means the box repeats itself (no edges), as in a bulk material; `False` means the box has edges.
- Some periodic boxes cannot be coloured by the method; the notebook then says so and uses open edges.

3.6 Active sites — `COORDS` and `SHAPE`

By default every site of the box carries a spin. To simulate a **shape** (a sphere, a disc, a wire…), first choose a coordinate system, then write the shape in it. The origin is the **centre of the box**, lengths are in lattice units and angles in radians.

`COORDS`	Names you can use	Example `SHAPE`
`"cartesian"`	`x`, `y`, `z`	`"(abs(x) <= 5) & (abs(y) <= 3)"` (a rectangle)
`"spherical"`	`r`, `theta`, `phi`	`"r <= 8"` (a sphere of radius 8)
`"cylindrical"`	`rho`, `phi`, `z`	`"(rho <= 10) & (abs(z) <= 2)"` (a disc)

- Join conditions with `&` (and), `|` (or) and `~` (not), and put each condition in parentheses. A ring: `"(rho <= 10) & (rho >= 6)"`.
- **To keep every site active, leave both empty:** `COORDS = ""` and `SHAPE = ""`.
- Make the box large enough to contain the shape, and use `PERIODIC = [False, False, False]` with a shape.

3.7 Temperatures and sweeps

Setting	Meaning	Typical value
`T_MAX`	first temperature (the hottest)	above the transition
`T_MIN`	last temperature (the coldest)	below the transition
`T_STEP`	distance between two temperatures	0.05 to 0.1
`THERMALIZATION`	sweeps to reach equilibrium at each temperature	1000 to 10000
`MEASUREMENT`	sweeps that are measured at each temperature	2000 to 20000
`MEASURE_EVERY`	measure once every this many sweeps	10
`REPLICAS`	independent copies of the system per state	32 to 128
`SEED`	seed of the random numbers; the same seed repeats the same run	any integer

One **sweep** means every spin gets one chance to change. More sweeps give smoother curves and take longer. On a GPU, more replicas cost little extra time per spin, so 64 or 128 is a good choice.

3.8 Output — `CONFIG_EVERY`, `OUTPUT_DIR`, `GPUS`

- `CONFIG_EVERY = 1` saves the configuration of every replica at **every** temperature; `2` at every second temperature, and so on. Configurations are the largest file (see section 5).
- `OUTPUT_DIR` is the name of the results folder.
- `GPUS = "all"` uses every GPU; `[0]` uses only the first one.

4. What you see while it runs

- **Cell 2** reports the hardware it found, for example `using 2 GPU(s): [0, 1]`, or that it runs on the CPU.
- **Cell 4** checks your parameters and prints how many sites are active and how large the configuration file will be. It **does not stop** the run because of the size: make sure there is enough disk.
- **Cell 5** runs the simulation and prints a line every few temperatures. With several GPUs, each line starts with the GPU that printed it, and the states are shared among the GPUs.
- **Cells 6 and 7** show a summary table and four plots: energy, |M|, Cv and χ against temperature.

As a guide, the default settings (Heisenberg on a 12 × 12 × 12 sc box, 32 replicas, 19 temperatures, 3000 sweeps per temperature) took about 20 seconds per state on a Kaggle T4.

5. The result files

Everything is written to the folder `simulation_output`:

File	Content	Shape
`magnetization.npy`	mean magnetization vector per site	[state, replica, temperature, 3]
`configurations.npy`	every spin of every replica	[state, replica, saved temperature, site, 3]
`sites.npy`	x, y, z position of every active site	[site, 3]
`results.npz`	temperatures, energy, $\langle E^2 \rangle$, $\langle \lvert M \rvert \rangle$, $\langle \lvert M \rvert^2 \rangle$, Cv, χ, state names	see below
`run.json`	every setting of the run, to repeat it	text

The last dimension `3` holds the x, y, z components; Ising uses only the first and XY the first two. Inside `results.npz`:

- `temperatures` — the list of temperatures;
- `energy`, `energy2`, `Mabs`, `M2` — $\langle E \rangle$, $\langle E^2 \rangle$, $\langle \lvert M \rvert \rangle$, $\langle \lvert M \rvert^2 \rangle$ per [state, replica, temperature];

- `E_site`, `M_site`, `Cv`, `chi` — averaged over replicas, per [state, temperature];
- `state_names` — the names of the states.

Cv and χ are computed per site from the fluctuations: $Cv = (\langle E^2 \rangle - \langle E \rangle^2) / (k_B T^2 N)$ and $\chi = N (\langle |M|^2 \rangle - \langle |M| \rangle^2) / (k_B T)$, with N the number of active sites.

Size of `configurations.npy` in bytes: states × replicas × saved temperatures × sites × 12. For the default run it is 38 MB; a large 3D box with 128 replicas and 200 temperatures can reach tens of GB, so increase `CONFIG_EVERY` in that case.

To open the files in Python:

```
import numpy as np
r = np.load("simulation_output/results.npz")
T, Cv = r["temperatures"], r["Cv"]                          # Cv[state, temperature]
M = np.load("simulation_output/magnetization.npy")          # M[state, replica,
↪  temperature, component]
C = np.load("simulation_output/configurations.npy", mmap_mode="r")   # read without
↪  loading it all
```

6. Ready-made examples

Copy the lines into cell 1, replacing the ones with the same name.

2D Ising model (exact transition at T = 2.269):

```
MODEL = "ising"; CRYSTAL = "square"; BOX = [64, 64]; PERIODIC = [True, True]
STATES = [dict(name="Ising", J={1: 1.0})]
COORDS = ""; SHAPE = ""
T_MAX = 3.5; T_MIN = 1.5; T_STEP = 0.05
```

Heisenberg sphere of radius 8:

```
MODEL = "heisenberg"; CRYSTAL = "sc"; BOX = [17, 17, 17]; PERIODIC = [False, False,
↪  False]
COORDS = "spherical"; SHAPE = "r <= 8"
```

Magnetic nanodisk with DMI, anisotropy and a perpendicular field (the paper's application):

```
MODEL = "heisenberg"; CRYSTAL = "sc"; BOX = [39, 39, 5]; PERIODIC = [False, False,
↪  False]
COORDS = "cylindrical"; SHAPE = "rho <= 18.3"
UNITS = "meV"
STATES = [dict(name="nanodisk", J={1: 1.0}, D=0.88, K1=0.177, Ks=0.139, K2=0.034,
↪  H=0.483, H_theta=0.0)]
T_MAX = 20.0; T_MIN = 0.1; T_STEP = 0.1
```

7. Reading the plots

- **|M|** is close to 0 when hot (disordered spins) and grows when cold (ordered spins).
- **Cv** and **χ** show a **peak** near the transition temperature. In a finite box the peak is rounded and slightly shifted; a larger box gives a sharper peak.
- If a peak sits at the first or last temperature, the transition is outside your range: widen `T_MAX`–`T_MIN`.
- Noisy curves: increase `MEASUREMENT` or `REPLICAS`. To locate a transition more precisely, use a smaller `T_STEP` around it.

8. Common messages and what to do

Message	What it means	What to do
`no usable GPU: running on the CPU`	no GPU, or one below compute capability 7.0	Kaggle: set the accelerator to GPU T4 x2; Colab: T4 GPU
`no admissible colouring for the periodic box …: using open boundaries`	that periodic box size is not supported	nothing (it continues with open edges), or change the box size
`D, K1, Ks and K2 need the Heisenberg model`	you gave D or K to Ising or XY	use `MODEL = "heisenberg"` or remove them
`unknown parameters ['Jx']`	a misspelled parameter name	use only J, D, K1, Ks, K2, H, H_theta, H_phi
`a shape needs its coordinate system`	`SHAPE` is filled and `COORDS` is empty	fill `COORDS`
`unknown coordinate system`	`COORDS` is misspelled	use cartesian, spherical or cylindrical
`the shape … contains no site of the box`	the shape is empty or outside the box	check the formula and the box size
`crystal sc needs a box of 3 sizes`	the box does not match the crystal	2D crystals take 2 sizes, 3D crystals take 3
`need T_max >= T_min > 0 and T_step > 0`	wrong temperatures	T_MAX above T_MIN, both positive
`state names must be distinct`	two states share a name	rename one
`SyntaxError`	a missing comma, quote or bracket in cell 1	compare the line with the examples

9. Reproducibility

The notebook writes its modules from text stored inside it and checks each one with a digital fingerprint (sha256), so it always runs the same code as the published repository `https://github.com/canamejoy/spin-colour-kernel`. The same parameters and the same `SEED` on the same kind of hardware give the same results. `run.json` stores every setting so that anyone can repeat the run.